

Elaborato Calcolo numerico 2023-2024

Autore: Davide Zhang
Matricola: 7073051
E-mail: davide.zhang@edu.unifi.it

Marzo 2024

-Esercizio 1

Dimostrare che :

$$\frac{25f(x) - 48f(x-h) + 36f(x-2h) - 16f(x-3h) + 3f(x-4h)}{12h} = f'(x) + o(h^4)$$

Per dimostrare la validità dell'uguaglianza procediamo a trovare gli sviluppi di Taylor centrati in x delle varie funzioni f:

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f^{(3)} + \frac{h^4}{24}f^{(4)} + o(h^5)$$
$$f(x-2h) = f(x) - 2hf'(x) + 2h^2f''(x) - \frac{4h^3}{3}f^{(3)}(x) + \frac{2h^4}{3}f^{(4)}(x) + o(h^5)$$
$$f(x-3h) = f(x) - 3hf'(x) + \frac{9h^2}{2}f''(x) - \frac{9h^3}{2}f^{(3)}(x) + \frac{27h^4}{8}f^{(4)}(x) + o(h^5)$$
$$f(x-4h) = f(x) - 4hf'(x) + 8h^2f''(x) - \frac{32h^3}{3}f^{(3)}(x) + \frac{32h^4}{3}f^{(4)}(x) + o(h^5)$$

A questo punto sostituendo e semplificando otteniamo che:

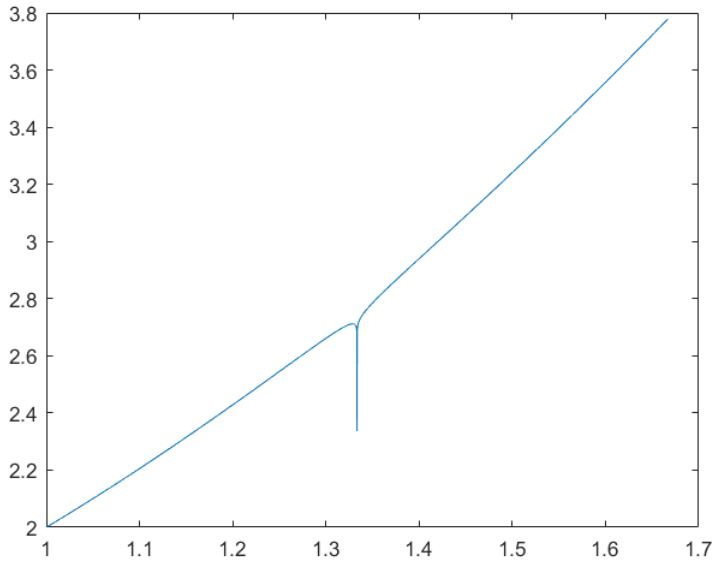
$$(25f(x) + (-48 + 36 - 16 + 3)f(x) + (48 - 72 + 48 - 12)hf'(x) + (-24 + 72 - 72 + 24)h^2f''(x)$$
$$+(8 - 48 + 72 - 32)h^3f^{(3)} + (-2 + 24 - 54 + 32)h^4f^{(4)} + o(h^5))/12h =$$
$$= \frac{25f(x) - 25f(x) + 12hf'(x) + o(h^5)}{12h} = \frac{12hf'(x) + o(h^5)}{12h} = f'(x) + o(h^4)$$

-Esercizio 2

Data f funzione reale con valori reali e ad asintoto verticale verso $-\infty$ in x=4/3:

$$f(x) = 1 + x^2 + \frac{log(|3(1-x)+1|)}{80} \qquad x \in [1, 5/3]$$

creare un grafico per rappresentarla su Matlab usando l'istruzione x=linspace(1,5/3,100001).



Come possiamo vedere il grafico presenta una sorta di picco verso il basso in corrispondenza dell'asintoto in $x = 4/3 \approx 1.33$. Data il numero finito di punti usati (sono 100003 inclusi i due punti all'estremo dell'intervallo), l'asintoto in $x = 4/3$ non viene rappresentata fedelmente e si ha solo un punto di minimo locale in (1.33333, 2.3359). Il minimo globale invece di non esistere per la presenza dell'asintoto verso meno infinito è situato nel punto (1,2).
(eseguire scriptExe2.m per ulteriori dettagli)

-Esercizio 3

Spiegare in modo esaustivo il fenomeno della cancellazione numerica. Fare un esempio che la illustri, spiegandone i dettagli.

Il fenomeno della cancellazione numerica è un fenomeno che si verifica quando si fa la somma di due valori con segno opposto: questo spesso risulta nella perdita di una o più cifre significative nel risultato. La causa della cancellazione numerica sta nella natura del numero di condizionamento k per la somma: dati x_1 *e* x_2 gli operandi dell'operazione, $k = \dfrac{|x_1| + |x_2|}{|x_1 + x_2|}$. Come si può vedere quando $x_1 \approx -x_2$, k crescerà tantissimo portando ad un'amplificazione importante dell'errore nel risultato che si osserva nella forma di perdite di cifre significative.

-Esercizio 4

Scrivere una function Matlab che implementi in modo efficiente il metodo di bisezione.

è riportata successivamente l'implementazione richiesta.

```
function [xstar, itnum]=metodoBisezione(f, a, b, tol)
%xstar=metodoBisezione(f, a, b, tol)
%Il metodo produce una stima di una radice per f in [a,b] se f assume segni
%discordi negli due estremi. La stima della radice ha un errore entro la
%tolleranza tol.
%Input:
%f-funzione per cui trovare la radice
%a-estremo sinistro dell'intervallo [a,b]
%b-estremo destro dell'intervallo [a,b]
%tol-tolleranza sull'errore
%Output:
%xstar-stima della radice
%itnum-numero di iterazioni avute per calcolare la radice
%Release 02-02-2024
if nargin==3
    tol=1e-3;
end
if nargin<3||nargin>4
        error("argomenti in numero errato");
end
if a>=b||tol<=0
    error("argomenti errati");
end
itnum=0;
fa=feval(f,a);
fb=feval(f,b);
if fa==0
    xstar=a;
    return
end
if fb==0
    xstar=b;
    return
end
if fa*fb>0
    error("intervallo errato f(a)*f(b)>0");
```

```
end
itmax=ceil(log2(b-a)-log2(tol));
for i=1:itmax
    xstar=(a+b)/2;
    itnum=itnum+1;
    fx=feval(f,xstar);
    if abs((fx*(b-a))/(fb-fa))<=tol
        return
    end
    if fa*fx<0
        b=xstar;
        fb=fx;
    else
        a=xstar;
        fa=fx;
    end
end
return
end
```

Listing 1: Metodo di bisezione

-Esercizio 5

Scrivere function Matlab distinte che implementino efficientemente i metodi di Newton e delle secanti per la ricerca degli zeri di una funzione f(x).

Riporto nelle pagine successive l'implementazione del metodo di Newton e di quello delle secanti.

```
function [xstar, itnum]=metodoNewton(f,df,x0,tol,itmax)
%xstar=metodoNewton(f,df,x0,tol,itmax)
%La funzione implementa il metodo di Newton per il calcolo della
    radice di
%una funzione f sufficientemente regolare.
%Input:
%f-funzione per cui trovare la radice
%df-derivata prima di f
%x0-punto iniziale
%tol-tolleranza
%itmax-numero massimo di iterazioni del metodo
%Output:
```

```
%xstar-approssimazione della radice di f entro la tolleranza tol o
    comunque
%prodotta con itmax iterazioni del metodo.
%itnum-numero di iterazioni avute per calcolare la radice
%Release 14-02-2024
if nargin==3
    tol=1e-3;
    itmax=1e3;
end
if nargin==4
    itmax=1e3;
end
if nargin<3||nargin>5
    error("numero di argomenti errato");
end
if tol<=0||itmax<=0
    error("argomenti errati");
end
flag=0;
itnum=0;
for i=1:itmax
    f0=feval(f,x0);
    df0=feval(df,x0);
    if f0==0
        xstar=x0;
        return
    end
    if df0==0
        warning("derivata prima uguale a zero, metodo interrotto");
        return
    end
    xstar=x0-f0/df0;
    itnum=itnum+1;
    if abs(xstar-x0)<=tol*(1+abs(xstar))
        flag=1;
        break;
    end
    x0=xstar;
end
if flag~=1
    warning("tolleranza non raggiunta");
end
return
end
```

Listing 2: Metodo di Newton

```
function [xstar, itnum]=metodoSecanti(f,x0,x1,tol,itmax)
%xstar=metodoSecanti(f, x0, x1, tol, itmax)
%Questo metodo effettua la ricerca di una radice per la funzione f
    partendo
%dagli punti x0 e x1. La radice sara' ricavata con una tolleranza tol
     oppure
%comunque ottenuta con un numero itmax di iterazioni.
%Input:
%f-funzione per cui si determina un'approssimazione di una radice
%x0-uno tra gli punti iniziali
%x1-uno tra gli punti iniziali
%tol-tolleranza
%itmax-numero massimo di iterazioni
%Output:
%xstar-l'approssimazione della radice di f con tolleranza tol
%itnum-numero di iterazioni avute per calcolare la radice
%Realease 02-02-2024
if nargin==3
    tol=1e-3;
    itmax=1e3;
end
if nargin==4
    itmax=1e3;
end
if nargin<3||nargin>5
    error("numero di argomenti errato");
end
if tol<=0||itmax<=0||x0==x1
    error("argomenti errati")
end
flag=0;
itnum=0;
fx0=feval(f,x0);
fx1=feval(f,x1);
if fx0==0
    xstar=x0;
    return
end
if fx1==0
    xstar=x1;
    return
end
for i=1:itmax
    if fx0==fx1
        if fx1==0
            xstar=x1;
            return
```

```
46          else
47              error("il metodo non converge");
48          end
49      end
50      xstar=(fx1*x0-fx0*x1)/(fx1-fx0);
51      itnum=itnum+1;
52      if abs(xstar-x1)<=tol*(1+abs(xstar))
53          flag=1;
54          break;
55      end
56      x0=x1;
57      x1=xstar;
58      fx0=fx1;
59      fx1=feval(f,xstar);
60  end
61  if flag~=1
62      warning("livello tol di tolleranza non e' stata raggiunta");
63  end
64  return
65  end
```

Listing 3: Metodo dei secanti

-Esercizio 6

Utilizzare le function dei precedenti esercizi per determinare una approssimazione della radice della funzione

$$f(x) = e^x - cos(x)$$

per $tol = 10^{-3}, 10^{-6}, 10^{-9}, 10^{-12}$, partendo da $x_0 = 1$ (e $x_0 = 0.9$ per il metodo delle secanti). Per il metodo di bisezione, usare l'intervallo di confidenza iniziale [-0.1, 1]. Tabulare i risultati, in modo da confrontare il costo computazionale di ciascun metodo.

Costruiamo la Tab. 1 in cui inseriamo le approssimazioni data dai vari metodi della radice di f in corrispondenza di ciascuna tolleranza.

Tab. 1	10^{-3}	10^{-6}	10^{-9}	10^{-12}
metodo di bisezione	$9,7656 * 10^{-4}$	$9,5367 * 10^{-7}$	$9,3132 * 10^{-10}$	$9,0949 * 10^{-13}$
metodo di Newton	$2,8423 * 10^{-9}$	$3,5748 * 10^{-17}$	$3,5748 * 10^{-17}$	$3,5748 * 10^{-17}$
metodo delle secanti	$1,1522 * 10^{-6}$	$2,0949 * 10^{-16}$	$2,0949 * 10^{-16}$	$-1,2557 * 10^{-17}$

Possiamo osservare che il metodo di Newton e quello delle secanti convergono molto più velocemente alla radice (situato in x=0) rispetto al metodo di bisezione che migliora via via la sua approssimazione in corrispondenza di ogni tolleranza. Inoltre osserviamo che il metodo di Newton per $tol = 10^{-6}$ è riuscito ad dare una approssimazione molto buona della radice (che sarebbe x=0): infatti Matlab valuta f uguale a 0 in corrispondenza di tale valore. Cosa simile è successa con il metodo delle secanti per lo stesso valore di tolleranza. Studiamo i costi dei metodi andando a vedere il numero totale di valutazioni di funzione per tolleranza nella Tab. 2.

Tab. 2	10^{-3}	10^{-6}	10^{-9}	10^{-12}
metodo di bisezione	9	19	29	39
metodo di Newton	5	6	6	6
metodo delle secanti	6	8	8	9

La tabella illustra il numero di iterazioni che i tre metodi hanno eseguito per arrivare al risultato. Per il metodo di bisezione il numero delle valutazioni di funzione è aumentato di due rispetto al numero di iterazioni (calcolo di f(a) e f(b)), per il metodo di Newton è il doppio (calcolo di f e della derivata di f per ogni iterazione), mentre per il metodo delle secanti è aumentato di uno (valutazione di f in x0). Possiamo quindi concludere che il costo minore di computazione sotto punto di vista del numero di valutazioni di funzione ce l'ha in questo caso il metodo delle secanti.
(per ulteriori dettagli vedere file esercizio6.txt o eseguire scriptexe6.m)

-Esercizio 7

Applicare gli stessi metodi e dati del precedente esercizio, insieme al metodo di Newton modificato, per la funzione

$$f(x) = e^x - cos(x) + sin(x) - x(x+2)$$

Tabulare i risultati, in modo da confrontare il costo computazionale e l'accuratezza di ciascun metodo. Commentare i risultati ottenuti.

Costruiamo una tabella per inserire le radici trovate con gli stessi dati iniziali dell'esercizio 6 usando come f la nuova funzione fornita.

Tab. 3	10^{-3}	10^{-6}	10^{-9}	10^{-12}
metodo di bisezione	0.037500	0.003125	0.001116	0.001116
metodo di Newton	0.003922	≈ -0.00008048	≈ -0.00008048	≈ -0.00008048
metodo delle secanti	0.005576	-0.001040	-0.001075	-0.001075
metodo di Newton modificato	≈ 0.00002602	≈ 0.00002602	≈ 0.00002602	≈ 0.00002602

In questo esercizio data l'irregolarità della funzione f ci sono stati molti problemi per il metodo di Newton e quello delle secanti: per il metodo di Newton non è stato possibile calcolare la stima della radice per le tolleranze 1e-6, 1e-9 e 1e-12 poiché la derivata prima della funzione f andava più velocemente verso lo zero rispetto alla funzione f, portando al blocco del metodo che richiede una derivata prima diversa da zero per il calcolo della stima successiva (la Tab. 3 mostra il valore di xstar restituito dalla funzione prima che si blocchi per la derivata prima nulla; Tab. 4, similmente, mostra il valore del numero di iterazioni al momento del blocco), mentre per il metodo delle secanti c'è stato un notevole aumento del numero dei cicli necessari senza un altrettanto miglioramento dell'accuratezza della stima. Per quanto Riguarda il metodo di Newton modificato che tiene conto della molteplicità della funzione (fExe7 ha molteplicità 5 in x=0), come il metodo di Newton ha avuto problemi relativi alla derivata prima, ma grazie alla sua maggiore velocità di convergenza fa un minor numero di iterazioni con risultati migliori. Il metodo di bisezione è il metodo più "stabile" con un costo che cresce proporzionalmente rispetto all'accuratezza ma non riesce ad avere un rapporto costo-accuratezza migliore

rispetto al metodo di Newton modificato.

Tab. 4	10^{-3}	10^{-6}	10^{-9}	10^{-12}
metodo di bisezione	3	5	31	32
metodo di Newton	25	41	41	41
metodo delle secanti	33	61	89	123
metodo di Newton modificato	2	2	2	2

(per ulteriori dettagli vedere file esercizio7.txt o eseguire scriptexe7.m)

-Esercizio 8

Scrivere una function Matlab “mialu” che che, data in ingresso una matrice A ed un vettore b, calcoli la soluzione del sistema lineare Ax = b con il metodo di fattorizzazione LU con pivoting parziale. Curare particolarmente la scrittura e l’efficienza della function, e validarla su un congruo numero di esempi significativi, che evidenzino tutti i suoi possibili output.

L’implementazione della funzione ”mialu”:

```
function x=mialu(A,b)
%x=mialu(A,b)
%mialu(A,b) calcola la soluzione al sistema lineare Ax=b, con A
    matrice
%quadrata nxn non singolare e b vettore dei termini noti nx1. La
%risoluzione del sistema passa prima per una fattorizzazione Lu con
%pivoting parziale di A.
%Input:
%A-matrice dei coefficienti. Deve essere non singolare e quadrata.
%b-colonna dei termini noti. Ha lo stesso numero di righe di A.
%Output:
%x-vettore soluzione nx1.
%release 07-02-2024.
[m,n]=size(A);
if m~=n||~iscolumn(b)||m~=length(b)
    error("Argomenti non compatibili")
```

```
16 end
17 tol=n*eps*norm(A,inf);
18 %fattorizzazione LU con pivoting di A
19 p=1:n;
20 for j=1:n
21     [mj,kj]=max(abs(A(j:n,j)));
22     if mj<=tol
23         error("Matrice A singolare");
24     end
25     kj=kj+j-1;
26     if kj>j
27         A([j kj],:)=A([kj j],:);
28         p([j kj])=p([kj j]);
29     end
30     A(j+1:n,j)=A(j+1:n,j)/A(j,j);
31     A(j+1:n,j+1:n)=A(j+1:n,j+1:n)-A(j+1:n,j)*A(j,j+1:n);
32 end
33 %risoluzione sistema PAx=Pb, in particolare Ly=Pb
34 x=b(p);
35 for j=1:n-1
36     x(j+1:n)=x(j+1:n)-x(j)*A(j+1:n,j);
37 end
38 %risoluzione sistema Ux=y
39 for j=n:-1:1
40     x(j)=x(j)/A(j,j);
41     x(1:j-1)=x(1:j-1)-x(j)*A(1:j-1,j);
42 end
43 return
44 end
```

Listing 4: funzione mialu, risolve i sistemi lineari attraverso la fattorizzazione LU con pivoting parziale

Successivamente riporto i risultati di alcuni esempi significativi di applicazione della funzione:

1. A matrice singolare, caso righe 1 e 2 uguali. La funzione riporta correttamente l'errore.
2. A matrice non quadrata. La funzione continua a riportare correttamente l'errore.
3. b non è una colonna. Come sopra.
4. argomenti compatibili. x calcolato risulta essere la soluzione corretta.
5. lunghezza b diversa da n. Errore riportato correttamente.

6. A matrice singolare, caso colonne 1 e 2 uguali. La funzione riporta correttamente l'errore.

(vedere i singoli casi eseguendo il file scriptExe8.m o esercizio8.txt)

-Esercizio 9

Scrivere una function Matlab "mialdl" che, dati in ingresso una matrice sdp A ed un vettore b, calcoli la soluzione del corrispondente sistema lineare utilizzando la fattorizzazione LDL^T. Curare particolarmente la scrittura e l'efficienza
della function, e validarla su un congruo numero di esempi significativi, che evidenzino tutti i suoi possibili output.

L'implementazione richiesta:

```
function x=mialdl(A,b)
%x=mialdl(A,b)
%mialdl calcola la soluzione del sistema lineare Ax=b, con A matrice
%quadrata fattorizzabile LDL^t (cioe' simmetrica e definita positiva)
     e b
%vettore dei termini noti compatibile. la risoluzione passa prima
%attraverso una fattorizzazione LDL^t di A.
%Input:
%A-matrice dei coefficienti. E' una matrice simmetrica e definita
    positiva.
%b-vettore dei termini noti. Deve avere lo stesso numero di righe di
    A.
%Output:
%x-vettore soluzione nx1.
%release 12-04-2024.
[m,n]=size(A);
if m~=n||~iscolumn(b)||m~=length(b)
    error("Argomenti incompatibili")
end
%fattorizzazione LDL^t di A
if ~issymmetric(A)
    error("Matrice A non e' s.d.p.");
end
if A(1,1)<=0
    error("Matrice A non e' s.d.p.");
end
A(2:n,1)=A(2:n,1)/A(1,1);
for j=2:n
```

```
26      v=(A(j,1:j-1).').*diag(A(1:j-1,1:j-1));
27      A(j,j)=A(j,j)-A(j,1:j-1)*v;
28      if A(j,j)<=0
29          error("Matrice A non e' s.d.p.");
30      end
31      A(j+1:n,j)=(A(j+1:n,j)-A(j+1:n,1:j-1)*v)/A(j,j);
32  end
33  %risoluzione sistema L*D*L^t*x=b, in particolare L*y=b
34  x=b;
35  for j=1:n-1
36      x(j+1:n)=x(j+1:n)-x(j)*A(j+1:n,j);
37  end
38  %risoluzione sistema D*L^t*x=y
39  for j=n:-1:1
40      x(j)=x(j)/A(j,j);
41      x(j)=x(j)-A(j+1:n,j).'*x(j+1:n);
42  end
43  return
44  end
```

Listing 5: funzione mialdl, risolve i sistemi lineari attraverso la fattorizzazione LDL^T

Qui sotto riporto i risultati di alcuni test di applicazione più significativi della funzione.

1. A non è quadrata. Errore riportato.
2. b lunghezza non compatibile. Errore riportato.
3. b non è una colonna. Come sopra.
4. A non simmetrica ma con elementi diagonali positivi. Come sopra.
5. A simmetrica ma non definita positiva. Come sopra.
6. Argomenti corretti. La soluzione x calcolata risulta essere giusta.

(per vedere in dettaglio le prove eseguire lo script scriptExe9 o vedere il file esercizio9.txt)

-Esercizio 10

Scrivere una function Matlab "miaqr" che data in ingresso la matrice A m × n, con m ≥ n = rank(A), ed un vettore b di lunghezza m, calcoli la soluzione del sistema lineare Ax = b nel senso dei minimi quadrati e, inoltre,

la norma, nr, del corrispondente vettore residuo. Curare particolarmente la scrittura e l'efficienza della function, e validarla su un congruo numero di esempi significativi, che evidenzino tutti i suoi possibili output.

l'implementazione richiesta:

```
function [x,nr]=miaqr(A,b)
%[x,nr]=miaqr(A,b)
%miaqr calcola la soluzione ai minimi quadrati e la norma del residuo del
%sistema lineare sovradeterminato Ax=b. A deve essere una matrice mxn con
%rango n<=m, mentre b e' un vettore mx1. La funzione restituisce un vettore
%soluzione di dimensione nx1. il calcolo della solzione passa prima
%attraverso una fattorizzazione QR di A, possibile se rank(A)=n<=m.
%Input:
%A-matrice mxn con rank(A)<=m.
%b-vettore mx1.
%Output:
%x-soluzione nel senso dei minimi quadrati. vettore nx1.
%nr-norma del residuo r=Ax-b.
%release 07-02-2024.
[m,n]=size(A);
if m<n||~iscolumn(b)||m~=length(b)
    error("Argomenti incompatibili");
end
tol=n*eps*norm(A,1);
%fattorizzazione QR di A
for j=1:n
    alfa=norm(A(j:m,j));
    if alfa<=tol
        error("Matrice A non ha rango massimo");
    end
    if A(j,j)>=0
        alfa=-alfa;
    end
    v1=A(j,j)-alfa;
    A(j+1:m,j)=A(j+1:m,j)/v1;
    A(j,j)=alfa;
    beta=-v1/alfa;
    A(j:m,j+1:n)= A(j:m,j+1:n)-beta*[1;A(j+1:m,j)]*([1;A(j+1:m,j)].'*.....
        A(j:m,j+1:n));
end
%risoluzione sistema Ax=b nel senso dei minimi quadrati
```

```
%calcolo di g
g=b;
for j=1:n
    g(j:m)=g(j:m)-(2/([1;A(j+1:m,j)].'*[1;A(j+1:m,j)]))...
        *([1;A(j+1:m,j)]*([1;A(j+1:m,j)].'*g(j:m)));
end
%risoluzione sistema R1*x=g1 (R1 prime n righe di R, g1 primi n
    elementi di
%g)
x=g(1:n);
for j=n:-1:1
    x(j)=x(j)/A(j,j);
    x(1:j-1)=x(1:j-1)-x(j)*A(1:j-1,j);
end
%calcolo della norma 2 del residuo
nr=norm(g(n+1:m));
return
end
```

Listing 6: funzione miaqr, risolve i sistemi lineari sovradeterminati nel senso dei minimi quadrati attraverso la fattorizzazione qr

test applicati:

1. A ha un maggior numero di colonne rispetto alle righe. Errore lanciato.
2. b ha un numero diverso di righe rispetto ad A. Errore lanciato.
3. A non ha rango massimo (colonna 1 e due uguali). Errore corrispondente osservato.
4. Argomenti corretti. Si osserva che x trovato soddisfa l'uguaglianza $\|ax-b\| = nr$; e sperimentando con valori diversi di x, il quadrato della norma di nr corrispondente al valore iniziale di x risulta sempre quella con valore minore.
5. A matrice quadrata. Risultato restituito è quello aspettato: x è una soluzione classica per il sistema lineare è la norma del residuo nr è nullo.
6. A matrice quadrata non singolare (riga 1 e riga 2 coincidenti). Segnalato che A non ha rango massimo.

(eseguire scriptExe10 o consultare file esercizio10.txt per informazioni ulteriori)

-Esercizio 11

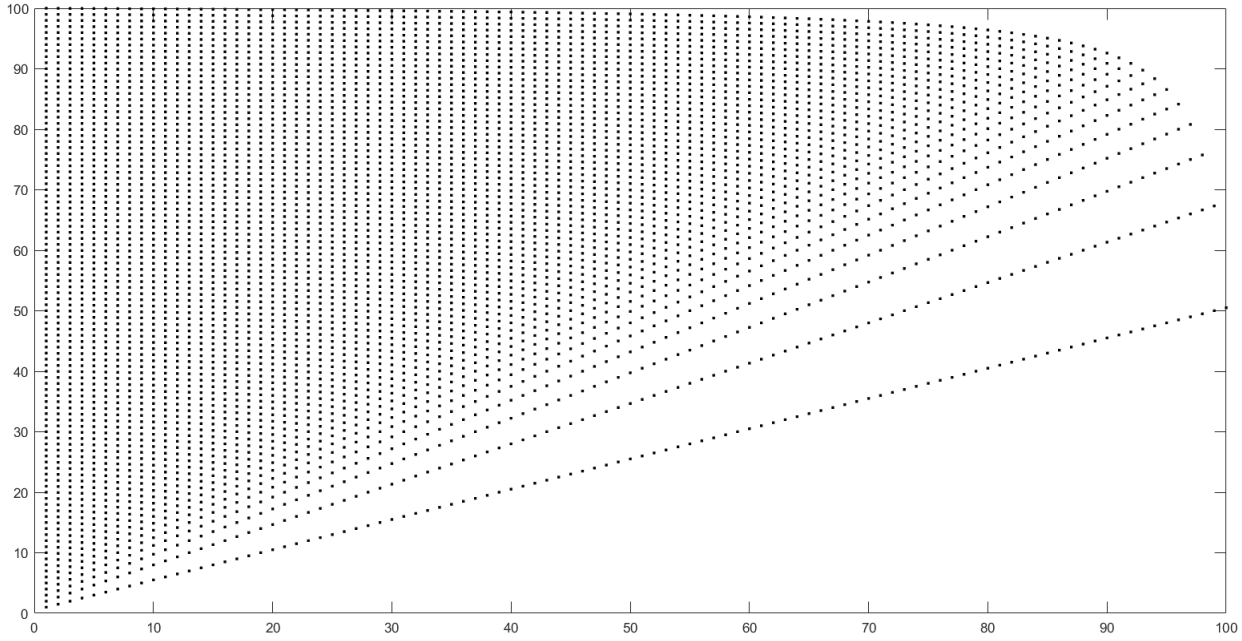
Risolvere i sistemi lineari di dimensione n $A_n x_n = b_n$ per n=1, ..., 15 con la struttura specificata. Utilizzare la funzione mialu, quindi tabulare e commentare l'accuratezza dei risultati ottenuti, dandone spiegazione esaustiva.

Costruiamo una tabella per inserire i risultati ottenuti usando la funzione mialu per risolvere i vari sistemi $A_n * x_n = b_n$ posti dall'esercizio. I risultati ottenuti sono abbastanza accurati rispetto al valore esatto $[1, 1, \ldots, 1]^T$, con deviazione visibile (per quattro cifre significative) solo dal quattordicesimo sistema della serie da risolvere. Questo può essere giustificato da un buon numero di condizionamento di sistema k(A) per i primi sistemi. Il numero di condizionamento di un sistema, calcolata da $\|A\| * \|A^-1\|$, fornisce una misura per l'amplificazione dell'errore relativo sulla soluzione dati gli "errori relativi" sui dati (A e b). A causa della natura della rappresentazione dei numeri frazionari in virgola mobile (sia precisione doppia che singola) i valori 11, 111, ... non possono essere rappresentati precisamente, e questo porta alla deviazione dal valore vero della soluzione (vettore contenenti n uni) per i sistemi con $n \geq 2$. Questi errori di rappresentazione, all'inizio nell'ordine di 10^-16, vengono poi amplificati fino ad essere errori importanti nei sistemi più grossi da risolvere che possono avere anche k(A) molto alto.
(eseguire script scriptExe11 per vedere i risultati ottenuti)

-Esercizio 12

Fattorizzare, utilizzando la function mialdlt, le matrici sdp fornite e graficare, in un unico grafico, gli elementi diagonali del fattore D, rispetto all'indice diagonale.

Costruiamo Un grafico a partire dagli elementi diagonali che si ottengono con la fattorizzazione LDL^T delle matrici fornite dall'esercizio: possiamo osservare che i punti risultanti hanno un andamento logaritmico decrescente per valori di indici crescenti, inoltre, per valori di n minori la curva diventa più "piatta". Per ogni valore di n, il grafico inizia nel punto (1, n) e termina vicino a (n, n/2).



(eseguire script scriptExe12.m per ottenere il grafico)

-Esercizio 13

Utilizzare la function "miaqr" per risolvere, nel senso dei minimi quadrati, il sistema lineare sovradeterminato specificato. Minimizzare la norma pesata del residuo; i pesi sono: $\omega_1 = \omega_2 = 0.5$, $\omega_3 = 0.75$, $\omega_4 = \omega_5 = 0.25$.
Dettagliare l'intero procedimento, calcolando, in uscita, anche ρ_ω.

La risoluzione del sistema lineare sovradeterminato Ax=b con obiettivo la minimizzazione della norma pesata ρ^2, avviene risolvendo il sistema sovradeterminato equivalente B*A*x=B*b, dove B è una matrice diagonale con elementi diagonali le radici quadrate dei pesi ω_i. Su Matlab andiamo prima a definire le matrici A e b, e un vettore v contenete i pesi ω_i. Siccome la somma dei ω_i deve essere uguale a 1, e nel nostro caso questo non succede, dividiamo gli elementi del vettore per 2.25 per rendere la somma tale (questo passaggio può essere anche saltato, il risultato non cambierebbe). Andiamo pertanto a costruire la matrice B dal vettore v appena citato usando la funzione diag() sul vettore v dopo aver fatto la radice quadrata sui suoi elementi. A questo punto risolviamo il sistema equivalente BAX=Bb usando la funzione "miaqr". La funzione restituisce una soluzione x e la corrispondente norma pesata wnr.
Verifichiamo la correttezza dei due valori andando a confrontare nr, la norma del residuo senza considerare i pesi, e wnr, norma calcolata considerando i pesi: calcoliamo prima il residuo per x da r=A*x-b, quindi deriviamo la norma pesata sommando i singoli elementi $r(i)^2$ del residuo moltiplicati per il loro corrispondente peso. Possiamo notare che la coppia (x,wnr) soddisfa il sistema lineare sovradeterminato iniziale Ax=b. (per svolgimento in Matlab e i risultati esatti eseguire script scriptExe13)

-Esercizio 14

Scrivere una function Matlab "newton", $[x, nit] = newton(fun, x0, tol, maxit)$, che implementi efficientemente il metodo di Newton per risolvere sistemi di equazioni nonlineari. Curare particolarmente il criterio di arresto. La seconda variabile, se specificata, ritorna il numero di iterazioni eseguite. Prevedere opportuni valori di default per gli ultimi due parametri di ingresso (rispettivamente, la tolleranza per il criterio di arresto, ed il massimo numero di iterazioni). La function fun deve avere sintassi: [f, jacobian]=fun(x), se il sistema da risolvere è f(x)=0.

Ecco l'implementazione richiesta:

```
1 function [x, nit]=newton(fun, x0, tol, maxit)
2 %[x, nit]=newton(fun, x0, tol, maxit)
3 %La funzione calcola la soluzione di un sistema di equazioni non
    lineare
```

```
%F(x)=0, dove F e' una funzione vettoriale R^m -> R^m, 0 e' una m-upla di
%zeri e x e' il vettore soluzione mx1.
%Input:
%fun-funzione matlab che oltre a restituire il valore di F(x) deve anche
%restituire F'(x), il valore della matrice Jacobiana, dato un m-upla x,
%attraverso la sintassi [f, jacobian]=f(x).
%x0-punto iniziale dell'iterazione per risolvere il sistema.
%tol-tolleranza prefissata.
%maxit-numero massimo di iterazioni consentite al metodo.
%Output:
%x-vettore soluzione mx1.
%nit-numero di iterazioni eseguite
%release 13-02-2024.
if nargin<2||nargin>4
    error("numero di argomenti errati");
end
if nargin==2
    tol=1e-3;
    maxit=1e3;
end
if nargin==3
    maxit=1e3;
end
if tol<=0||maxit<=0
    error("argomenti errati");
end
x0=x0(:);
flag=0;
nit=0;
x=x0;
isSing=0;
for i=1:maxit
    [Fx, dFx]=feval(fun,x);
    [m,n]=size(dFx);
    l=length(Fx);
    %fattorizzazione LU di dFx
    if m~=n||m~=l
        error("funzione fun errata");
    end
    for j=1:n
        if dFx(j,j)==0
            isSing=1;
            break;
        end
```

```
48          dFx(j+1:n,j)=dFx(j+1:n,j)/dFx(j,j);
49          dFx(j+1:n,j+1:n)=dFx(j+1:n,j+1:n)-dFx(j+1:n,j)*dFx(j,j+1:n);
50      end
51      if isSing==1
52          warning("fallimento fattorizzazione LU");
53          return
54      end
55      %risoluzione del sistema lineare
56      %risoluzione sistema lineare Ly=b
57      y=-Fx;
58      for j=1:n
59          y(j)=y(j)-dFx(j,1:j-1)*y(1:j-1);
60      end
61      %risoluzione sistema lineare Ux=y
62      for j=n:-1:1
63          y(j)=y(j)-dFx(j,j+1:n)*y(j+1:n);
64          y(j)=y(j)/dFx(j,j);
65      end
66      dx=y;
67      if norm(dx./(1+abs(x)),inf)<=tol
68          flag=1;
69          break;
70      end
71      x=x+dx;
72      nit=nit+1;
73  end
74  if flag~=1
75      warning("livello di tolleranza non raggiunta");
76  end
77  return
78  end
```

Listing 7: funzione newton, usa la fattorizzazione LU per risolvere i sistemi lineari intermedi

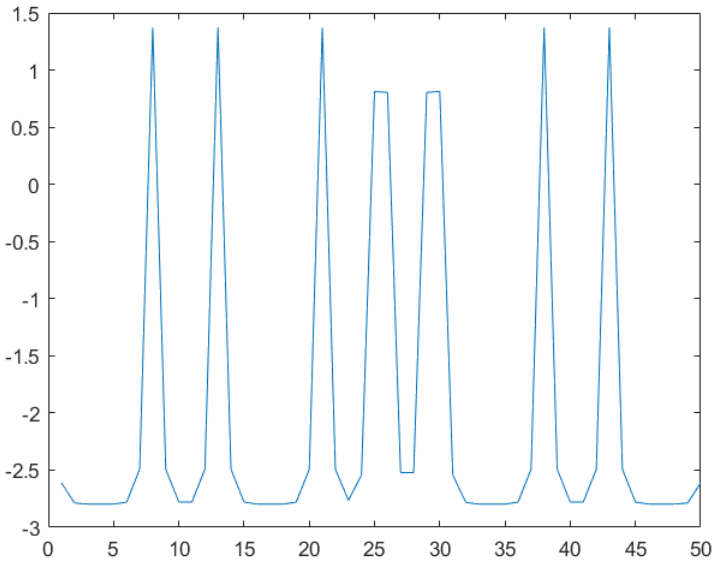
-Esercizio 15

Usare la function del precedente esercizio per risolvere, a partire dal vettore iniziale nullo, il sistema non-lineare derivante dalla determinazione del punto stazionario della funzione specificata. utilizzando tolleranze tol = 1e-3, 1e-8, 1e-13 (le function cos e exp sono da intendersi in modo vettoriale). Graficare la soluzione e tabulare in modo conveniente i risultati ottenuti.

Calcolando le derivate prime parziali della funzione scalare iniziale rispetto ai suoi vari componenti otteniamo 50 funzioni: queste fanno parte della funzione vettoriale di cui vogliamo trovare lo zero con il metodo di Newton. Vengono quindi ordinate secondo la componente di derivazione parziale per ottenere $F : R^{50} \to R^{50}$. Da qui viene costruito la matrice Jacobiana corrispondente calcolando per ogni funzione f_i tutte le sue derivate parziali e inserendo i risultati in una matrice 50×50 (nella riga i-esima seguendo l'ordine dei componenti). Ho quindi costruito la funzione "fExe15" che implementa entrambe le funzionalità di calcolo. L'applicazione di "newton" su "fExe15" ha portato ai risultati mostrati dallo script (eseguire scriptExe15). I numeri di iterazioni eseguite per le varie tolleranze sono:

Tab. 5	10^{-3}	10^{-8}	10^{-13}
num. iterazioni	699	700	701

Possiamo notare che ci sono poche differenze per quanto riguarda il numero delle iterazioni per le tre tolleranze: per tolleranze maggiori come 10^{-2} o 0.1 il numero di iterazioni è di un'ordine di grandezza minore ma le radici trovate non sono molto buone (la funzione F in queste radici assume valori molto grossi in alcuni componenti). Se invece di usare la fattorizzazione LU si usasse quella con pivoting parziale il metodo di Newton si fermerebbe alle prime 3 iterazioni per matrice Jacobiana singolare (il confronto che include la norma fallisce); se si sostituisse questo controllo la funzione restituirebbe, dopo aver esaurito il numero di iterazioni a disposizione, un vettore con tutti gli elementi a NaN. La causa di questo fenomeno può essere l'elevato numero di condizionamento delle matrici Jacobiane che si calcolano via via in questa versione. I vettori restituiti con la versione con fattorizzazione LU in confronto sono approssimazioni molto buone della particolare radice di F. Di seguito è un grafico costruito assegnando al valore di ogni componente della soluzione restituita per $tol = 10^{-13}$ al corrispondente indice:



-Esercizio 16

Costruire una function, lagrange.m, avente la stessa sintassi della function spline di Matlab, che implementi, in modo vettoriale, la forma di Lagrange del polinomio interpolante una funzione.

Funzione "lagrange" implementata:

```
function s=lagrange(x,y,xq)
%s=lagrange(x,y,xq)
%lagrange restituisce in corrispondenza del vettore xq delle query
%points il vettore s dei valori interpolati ai query points della
    stessa
%dimensione in base alla funzione interpolante calcolata in forma di
%Lagrange dai valori nei vettori x e y.
%Input:
%x-vettore delle ascisse degli interpoli, i valori delle ascisse
    devono
%essere distinte.
%y-vettore delle ordinate degli interpoli.
%xq-vettore query points, contenente le ascisse dei punti di
    interesse che
```

```
%devono essere ordinate in maniera crescente.
%Output:
%s-vettore risultato, contiene i valori restituiti della funzione
%interpolante in corrispondenza dei query points.
%release 22-04-2024.
if nargin~=3
    error("numero di argomenti errato");
end
lx=length(x);
ly=length(y);
lq=length(xq);
if lx~=ly
    error("argomenti errati");
end
s=zeros(lq,1);
for i=1:lq
    for j=1:lx
        temp=1;
        for k=1:lx
            if k~=j
                if x(k)==x(j)
                    error("vettore x errato");
                end
                temp=temp*((xq(i)-x(k))/(x(j)-x(k)));
            end
        end
        s(i)=s(i)+temp*y(j);
    end
end
return
end
```

Listing 8: funzione lagrange, calcola i valori che la funzione interpolante in forma di Lagrange assume nei punti richiesti

-Esercizio 17

Costruire una function, newton.m, avente la stessa sintassi della function spline di Matlab, che implementi, in modo vettoriale, la forma di Newton del polinomio interpolante una funzione.

Funcione “newton” implementata:

```
function s=newton2(x,y,xq)
%s=newton2(x,y,xq)
%newton restituisce in corrispondenza del vettore xq delle query
%points il vettore s dei valori interpolati ai query points della
    stessa
%dimensione in base alla funzione interpolante calcolata in forma di
%Newton dai valori nei vettori x e y.
%Input:
%x-vettore delle ascisse degli interpoli, i valori delle ascisse
    devono
%essere distinte.
%y-vettore delle ordinate degli interpoli.
%xq-vettore query points, contenente le ascisse dei punti di
    interesse che
%devono essere ordinate in maniera crescente.
%Output:
%s-vettore risultato, contiene i valori restituiti della funzione
%interpolante in corrispondenza dei query points.
%release 22-04-2024.
if nargin~=3
    error("numero di argomenti errato");
end
lx=length(x);
ly=length(y);
lq=length(xq);
if lx~=ly
    error("argomenti errati");
end
s=zeros(lq,1);
%calcolo delle differenze divise
for i=1:lx-1
    for j=lx:-1:i+1
        if x(j)==x(j-i)
            error("argomenti errati");
        end
        y(j)=(y(j)-y(j-1))/(x(j)-x(j-i));
    end
end
%calcolo dei valori del polinomio interpolante nella forma di Newton
    ai
%punti di interesse
for i=1:lq
    for j=1:lx
        temp=1;
        for k=1:j-1
            temp=temp*(xq(i)-x(k));
        end
```

```
44          s(i)=s(i)+temp*y(j);
45      end
46  end
47  return
48  end
```

Listing 9: funzione newton, calcola i valori che la funzione interpolante in forma di Newton restituisce nei punti di interesse

-Esercizio 18

Costruire una function, hermite.m, avente sintassi yy=hermite(xi, fi, f1i, xx) che implementi, in modo vettoriale, il polinomio interpolante di Hermite.

L'mplementazione richiesta:

```
1  function yy=hermite(xi,fi,f1i,xx)
2  %yy=hermite(xi,fi,f1i,xx)
3  %hermite restituisce in corrispondenza dei valori nel vettore xx
       delle
4  %query points il vettore yy dei valori interpolati ai query points in
        base
5  %al polinomio interpolante di Hermite.
6  %Input:
7  %xi-vettore delle ascisse degli interpoli, i valori delle ascisse
       devono
8  %essere distinte.
9  %fi-vettore delle ordinate degli interpoli.
10 %f1i-vettore delle derivate prime della funzione interpolata in
11 %corrispondenza degli interpoli.
12 %xx-vettore query points, contenente le ascisse dei punti di
       interesse che
13 %devono essere ordinate in maniera crescente.
14 %Output:
15 %yy-vettore risultato, contiene i valori restituiti della funzione
16 %interpolante in corrispondenza dei query points.
17 %release 22-04-2024.
18 if nargin~=4
19     error("numero di argomenti errato");
20 end
21 lx=length(xi);
22 ly=length(fi);
23 ldy=length(f1i);
```

```
lq=length(xx);
if lx~=ly||lx~=ldy
    error("argomenti errati");
end
yy=zeros(lq,1);
%calcolo delle differenze divise
rx=zeros(2*lx,1);
rx(1)=xi(1);
rx(2*lx)=xi(lx);
dd=zeros(2*lx,1);
dd(1)=fi(1);
dd(2*lx)=f1i(lx);
for i=1:lx-1
    if xi(i+1)==xi(i)
        error("argomenti errati");
    end
    rx(2*i)=xi(i);
    rx(2*i+1)=xi(i+1);
    dd(2*i)=f1i(i);
    dd(2*i+1)=(fi(i+1)-fi(i))/(xi(i+1)-xi(i));
end
for i=2:2*lx-1
    for j=2*lx:-1:i+1
        if rx(j)==rx(j-i)
            error("argomenti errati");
        end
        dd(j)=(dd(j)-dd(j-1))/(rx(j)-rx(j-i));
    end
end
%calcolo dei valori del polinomio interpolante di Hermite nei
%punti di interesse
for i=1:lq
    for j=1:lx*2
        temp=1;
        for k=1:j-1
            if mod(k,2)~=0
                temp=temp*(xx(i)-xi(ceil(k/2)));
            else
                temp=temp*(xx(i)-xi(k/2));
            end
        end
        yy(i)=yy(i)+temp*dd(j);
    end
end
return
```

```
69 end
```

Listing 10: funzione hermite, calcola i valori che il polinomio interpolante di Hermite assume in corrispondenza dei query points

-Esercizio 19

Considerando la seguente base di Newton

$$\omega_i(x) = \prod_{j=0}^{i-1}(x - x_j) \qquad i = 0, \ldots, n$$

con $x_0, \ldots, x_n$ ascisse date (non necessariamente distinte tra loro), ed un polinomio rappresentato rispetto a tale base

$$p(x) = \sum_{i=0}^{n} a_i \omega_i(x)$$

Derivare una modifica dell' algoritmo di Horner per calcolarne efficientemente la derivata prima.

Dall'algoritmo di Horner sappiamo che possiamo scomporre il polinomio p(x) in questo modo:

$$p(x) = \sum_{i=0}^{n} a_i \omega_i(x) = a_0 + (x - x_0)(a_1 + (x - x_1(a_2 + (x - x_2)(a_3 + \ldots))))$$

Effettutando la derivata prima di questa espressione otteniamo che p(x) può essere riscritto come:

$$\begin{aligned} p(x) = 1 * [a_1 + (x - x_1)(a_2 + (x - x_2)(a_3 + \ldots (x - x_{n-1})a_n))] + \\ (x - x_0) * ([a_2 + (x - x_2)(a_3 + \ldots (x - x_{n-1})a_n)] + \\ (x - x_1) * ([a_3 + \ldots ((x - x_{n-1})a_n)] + \\ \ldots \\ (x - x_{n-3}) * ([a_{n-1} + ((x - x_{n-1})a_n] + \\ (x - x_{n-2}) * [a_n])) \end{aligned}$$

E quindi si ottiene:

$$
\begin{aligned}
p(x) = [a_1 + (x - x_1)(a_2 + (x - x_2)(a_3 + \ldots (x - x_{n-1})a_3))] + \\
(x - x_0) \qquad\qquad [a_2 + (x - x_2)(a_3 + \ldots (x - x_{n-1})a_n)] + \\
(x - x_1)(x - x_0) \qquad\qquad [a_3 + \ldots (x - x_{n-1})a_n] + \\
\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \ldots \\
(x - x_{n-3})(x - x_{n-4}) * \cdots * (x - x_0) \quad [a_{n-1} + (x - x_{n-1})a_n] + \\
(x - x_{n-2})(x - x_{n-3})(x - x_{n-4}) * \cdots * (x - x_0) \qquad [a_n]
\end{aligned}
$$

Da qui si ha la seguente funzione Matlab:

```
function dfx0=hornerDer(x,a,x0)
%dfx0=hornerDer(x,a,x0)
%hornerDer e' un'algoritmo basato su quello di Horner per calcolare
    la
%derivata prima di un polinomio rappresentato attraverso le basi di
    Newton.
%Input:
%x-vettore delle ascisse usate per la base di Newton
%a-coefficienti del polinomio
%x0-vettore dei punti su cui vogliamo valutare il polinomio
%Output:
%dfx0-vettore delle derivate prime calcolate in corrispondenza delle
%ascisse di interesse
%release 22-04-2024
lx=length(x);
la=length(a);
lr=length(x0);
if lx~=la
    error("dati errati");
end
t=zeros(lx-1,lr);
t(lx-1,:)=a(lx)*ones(1,lr);
for i=lx-2:-1:1
    for j=1:lr
    t(i,j)=t(i+1,j)*(x0(j)-x(i+1))+a(i+1);
    end
end
dfx0=t(1,:);
for i=2:lx-1
    for j=1:lr
        t(i:lx-1,j)=t(i:lx-1,j)*(x0(j)-x(i-1));
    end
    dfx0=dfx0+t(i,:);
```

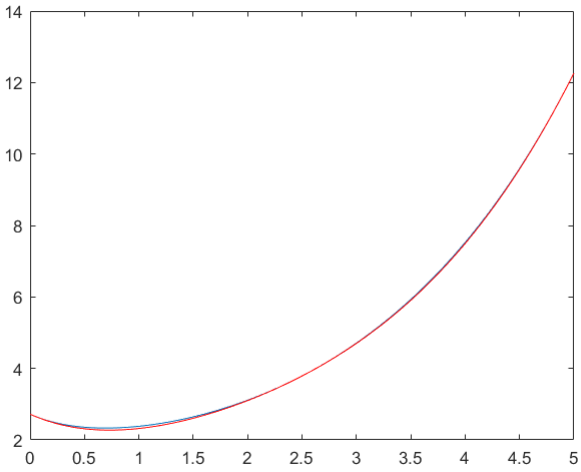
```
32 end
33 return
34 end
```

Listing 11: algoritmo di Horner modificato per il calcolo della derivata prima di un polinomio espresso in base di Newton

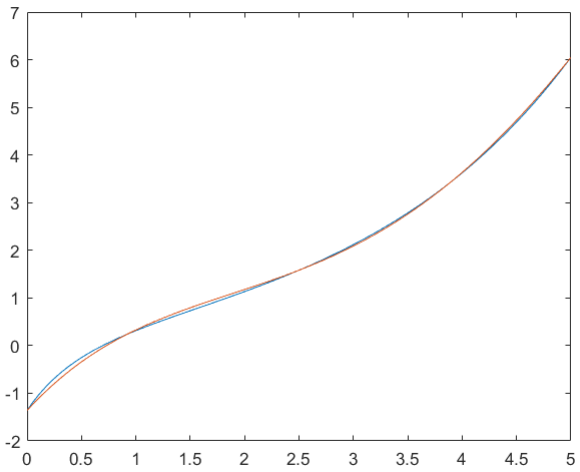
-Esercizio 20

Utilizzando le function degli esercizi 18 e 19, calcolare il polinomio interpolante di Hermite per la funzione $f(x) = exp(x/2 + exp(-x))$ sulle ascisse$\{0, 2.5, 5\}$. Graficare il grafico della funzione interpolanda e del polinomio interpolante nell'intervallo [0, 5], e quello della derivata prima della funzione interpolanda, e della derivata prima del polinomio interpolante, verificando graficamente le condizioni di interpolazione per entrambi.

Ecco i due grafici ottenuti (eseguire scriptExe20): le curve rosse rappresentano il polinomio interpolante e la sua derivata mentre quelle blu la funzione interpolata e la sua derivata:



Graf. 1: raffigura la funzione f(x) e il suo polinomio interpolante p(x)



Graf. 2: raffigura l'andamento della derivata prima di f(x) e di p(x)

Possiamo verificare che le condizioni di interpolazione vengono effettivamente verificate per x=0, 2.5 e 5.

-Esercizio 21

Costruire una function Matlab che, specificato in ingresso il grado n del polinomio interpolante, e gli estremi dell'intervallo [a, b], calcoli le corrispondenti ascisse di Chebyshev.

```
function x=ceby(a,b,n)
%x=ceby(a,b,n)
%calcola le ascisse di Chebychev per il polinomio interpolante di
    grado n
%sull'intervallo [a,b]
%Input:
%a,b-estremi dell'intervallo
%n-grado del polinomio interpolante
%Output:
%x-vettore delle ascisse di Chebychev in uscita
%release 23-04-2024
if nargin<3||n<=0||a>b
    error("argomenti errati");
end
x=cos((2*n+1:-2:1)*pi/(2*n+2));
```

```
15 x=(a+b)/2+x*((b-a)/2);
16 return
17 end
```

Listing 12: questa function permette di determinare le ascisse di Chebychev in un determinato intervallo dato per una funzione interpolante di grado n

-Esercizio 22

Costruire una function Matlab, con sintassi ll=lebesgue(a,b,nn,type), che approssimi la costante di Lebesgue per l'interpolazione polinomiale sull'intervallo [a,b], per i polinomi di grado specificato nel vettore nn, utilizzando ascisse equidistanti, se type=0, o di Chebyshev, se type=1 (utilizzare 10001 punti equispaziati nell'intervallo [a,b] per ottenere ciascuna componente di ll). Graficare opportunamente i risultati ottenuti, per nn=1:100, utilizzando [a,b]=[0,1] e [a,b]=[-5,8]. Commentare i risultati ottenuti.

Eseguendo scriptExe22 vengono generate 4 finestre, una per ogni grafico. Possiamo notare che la scelta degli intervalli non influenza minimamente il valore della costante di Lebesgue calcolata, il quale cresce in maniera esponenziale nel caso di ascisse equidistanti e logaritmicamente nel caso di ascisse di Lebesgue; riporto qui entrambi nel caso dell'intervallo [0,1].

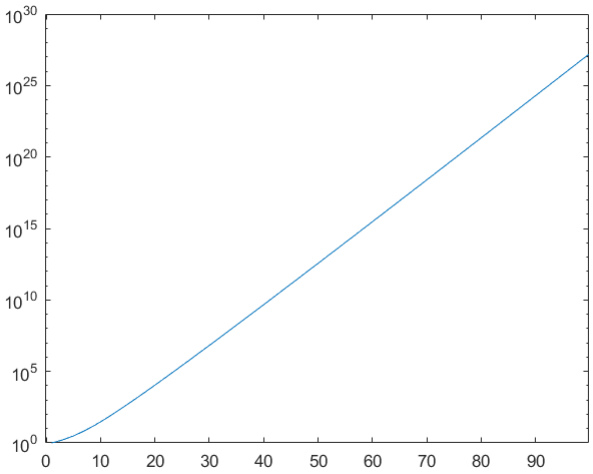


Fig. 1: caso ascisse equidistanti. È stata scelta una scala logaritmica sull'asse delle ordinate.

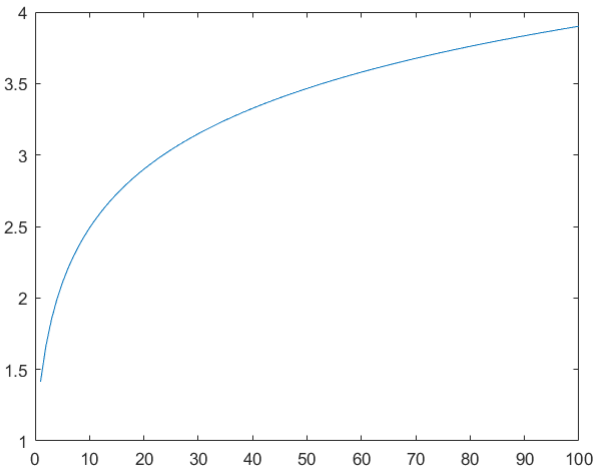


Fig. 2: caso ascisse di Chebyshev.

L'implementazione richiesta:

```
function ll=lebesgue(a,b,nn,type)
%ll=lebesgue(a,b,nn,type)
%lebesgue restituisce una stima per la costante di Lebesgue nell'
    intervallo
%[a,b] per i polinomi interpolanti di grado specificato nel vettore
    nn.
%Input:
%a,b-estremi dell'intervallo di calcolo
%nn-vettore contenente i gradi dei polinomi per cui si stima la
    costante di
%Lebesgue
%type-usato per specificare il tipo di ascisse usate nella stima
    della
%costante: se type==0 ascisse equidistanti, se type==1 ascisse di
%Chebychev.
%Output:
%ll-vettore delle stime della costante di Lebesgue per ogni grado
%richiesto.
if nargin<3||nargin>4
    error("numero di argomenti errato");
end
if nargin==3
    type=0;
end
if a>b||type~=0&&type~=1
```

```
    error("argomenti errati");
end
n=length(nn);
x=linspace(a,b,10001).';
%caso ascisse equidistanti
if type==0
    for i=1:n
        gr=nn(i);
        amp=(b-a)/gr;
        xi=zeros(gr+1,1);
        for j=1:gr+1
            xi(j)=(j-1)*amp+a;
        end
        tempS=zeros(10001,1);
        for j=1:gr+1
            tempP=ones(10001,1);
            for k=1:gr+1
                if k~=j
                    tempP=tempP.*((x-xi(k))./(xi(j)-xi(k)));
                end
            end
            tempS=tempS+abs(tempP);
        end
        nn(i)=max(tempS);
    end
    %caso ascisse Chebychev
else
    for i=1:n
        gr=nn(i);
        xi=cos((2*gr+1:-2:1)*pi/(2*gr+2));
        xi=(a+b)/2+xi*((b-a)/2);
        tempS=zeros(10001,1);
        for j=1:gr+1
            tempP=ones(10001,1);
            for k=1:gr+1
                if k~=j
                    tempP=tempP.*((x-xi(k))./(xi(j)-xi(k)));
                end
            end
            tempS=tempS+abs(tempP);
        end
        nn(i)=max(tempS);
    end
end
ll=nn;
return
```

```
68 end
```

Listing 13: lebesgue calcola una stima per la costante di Lebesgue

-Esercizio 23

Utilizzando le function degli esercizi 16 e 17, graficare (in semilogy) l'andamento errore di interpolazione (utilizzare 10001 punti equispaziati nell'intervallo per ottenerne la stima) per la funzione di Runge, utilizzando le ascisse di Chebyshev, per i polinomi interpolanti di grado n=2:2:100. Commentare i risultati ottenuti.

Ecco i grafici ottenuti:

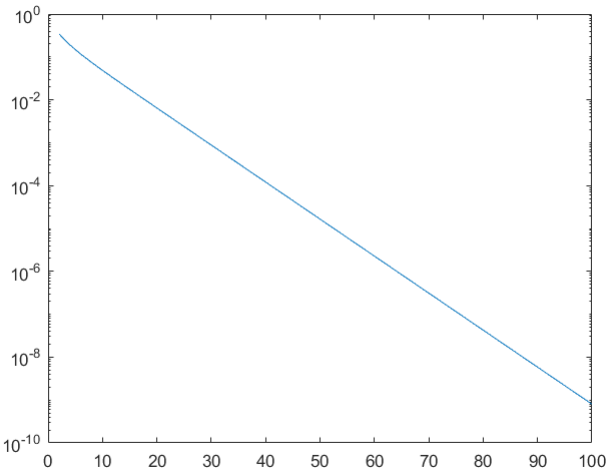


Fig. 1: Caso polinomio interpolante in forma di Lagrange.

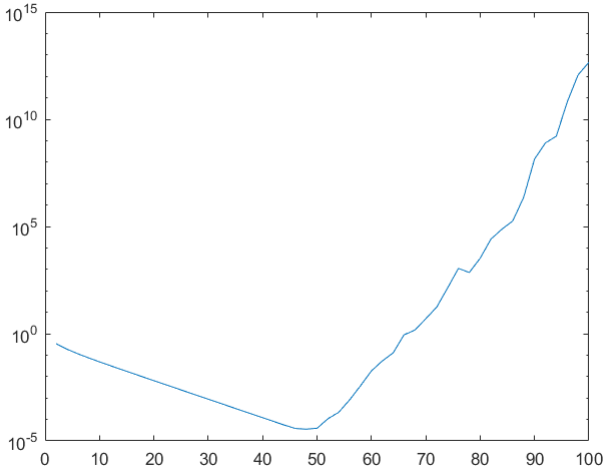


Fig. 2: Caso polinomio interpolante in forma di Newton.

Possiamo osservare che per entrambi i tipi di polinomi interpolanti l'errore di interpolazione tende inizialmente in modo esponenziale a zero, tuttavia nel caso del polinomio interpolante di Newton per gradi da 50 in poi l'errore inizia a crescere in modo esponenziale fino ad arrivare ad un valore di circa 10^{14} per n=100. Il polinomio di Lagrange si comporta invece in maniera più regolare e l'errore assoluto medio decresce in maniera esponenziale al crescere del grado del polinomio interpolante fino alla fine del grafico.

-Esercizio 24

Costruire una function, spline0.m, avente la stessa sintassi della function spline di Matlab, che calcoli la spline cubica interpolante naturale i punti (xi,fi).

Ecco l'implementazione richiesta:

```
function s=spline0(x,y,xq)
%s=spline0(x,y,xq)
%spline0 calcola la spline cubica naturale per una funzione f(x) dati
    i
%vettori x e y, rispettivamente vettore delle ascisse della
   partizione e
%vettore delle corrispondenti ordinate assunte.
%Input:
```

```
%x-vettore delle ascisse (devono essere ordinate in modo crescente)
%y-vettore dei valori della funzione alle corrispondenti ascisse
%xq-vettore dei query-points, ordinato in modo crescente
%Output:
%s-valori interpolati ai query-points.
%release 24-04-2024
if nargin~=3
    error("numero di argomenti errati");
end
lx=length(x);
ly=length(y);
if lx~=ly
    error("lunghezza vettore x e y differenti");
end
lq=length(xq);
if xq(1)<x(1)||xq(lq)>x(lx)
    error("xq errato");
end
%calcolo delle differenze divise
if ~iscolumn(y)
    y=y.';
end
dd=y;
for i=1:2
    for j=lx:-1:i+1
        if x(j)==x(j-i)
            error("ascisse non distinte");
        end
        dd(j)=(dd(j)-dd(j-1))/(x(j)-x(j-i));
    end
end
dd=dd(3:lx);
%calcolo delle csi_i e phi_i
csi=zeros(lx-3,1);
phi=zeros(lx-3,1);
for i=1:lx-3
    temp=x(i+2)-x(i+1);
    csi(i)=temp/(x(i+2)-x(i));
    phi(i)=temp/(x(i+3)-x(i+1));
end
%calcolo m_i
diag=ones(lx-2,1)*2;
m=6*dd;
for i=1:lx-3
    phi(i)=phi(i)/diag(i);
    diag(i+1)=diag(i+1)-phi(i)*csi(i);
    m(i+1)=m(i+1)-phi(i)*m(i);
```

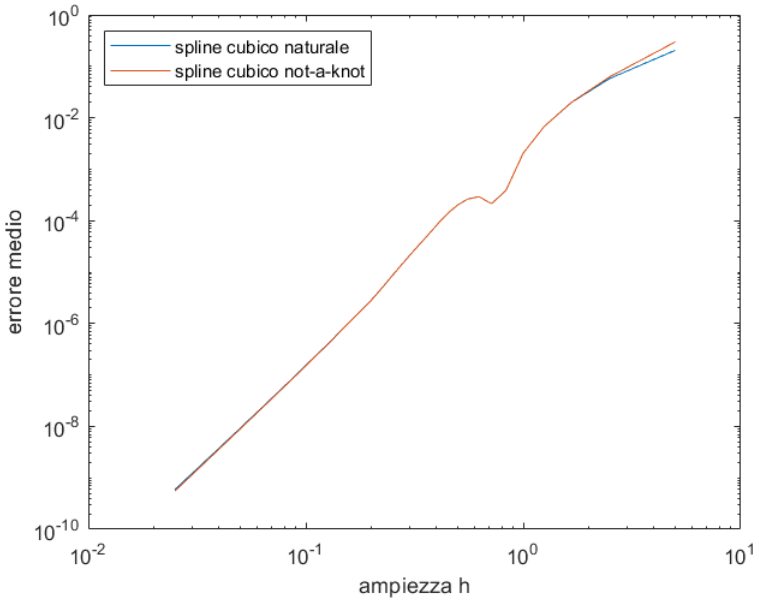
```
54 end
55 m(lx-2)=m(lx-2)/diag(lx-2);
56 for i=lx-3:-1:1
57     m(i)=(m(i)-csi(i)*m(i+1))/diag(i);
58 end
59 m=[0;m;0];
60 %calcolo vettore s
61 s=zeros(lq,1);
62 ind=1;
63 for i=1:lq
64     while x(ind)<=xq(i)&&ind<lx
65         ind=ind+1;
66     end
67     qi=(y(ind)-y(ind-1))/(x(ind)-x(ind-1))-...
68         (x(ind)-x(ind-1))*(m(ind)-m(ind-1))/6;
69     ri=y(ind-1)-(x(ind)-x(ind-1))^2*m(ind-1)/6;
70     s(i)=((xq(i)-x(ind-1))^3*m(ind)+(x(ind)-xq(i))^3*m(ind-1))/...
71         (6*(x(ind)-x(ind-1)))+...
72         qi*(xq(i)-x(ind-1))+ri;
73 end
74 return
75 end
```

Listing 14: spline0 calcola lo spline cubico naturale per una funzione in un intervallo dato una partizione dello stesso

-Esercizio 25

Graficare, usando il formato loglog, l'errore di approssimazione degli spline interpolanti naturale e not-a-knot per approssimare la funzione di Runge sull'intervallo [-10, 10], utilizzando una partizione uniforme rispetto alla distanza h = 20/n tra le ascisse e con numero di punti nella partizione n=4:4:800. Utilizzare 10001 punti equispaziati nell'intervallo [-10, 10] per ottenere la stima dell'errore. Che tipo di decrescita si osserva?

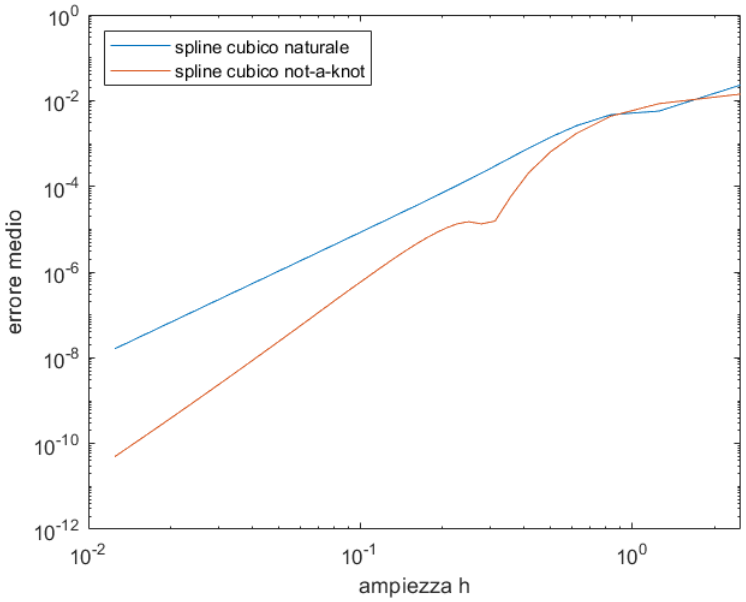
Possiamo osservare che gli errori di approssimazione medi per i due tipi di spline seguono un andamento quasi identico: hanno in generale una decrescita esponenziale verso lo zero. Lo spline cubico naturale inizia con un errore leggermente più piccolo ma viene presto raggiunto da quello dello spline not-a-knot, i due errori quindi seguono lo stesso andamento fino a raggiungere un numero di circa 300 punti nella partizione dopodiché iniziano a divergersi: l'errore dello spline not-a-knot inizia ad assumere valori un po' più piccoli.



-Esercizio 26

Graficare, utilizzando il formato loglog, l'errore di approssimazione utilizzando le spline interpolanti naturale e not-a-knot per approssimare la funzione di Runge sull'intervallo [0,10], utilizzando una partizione uniforme rispetto alla distanza h = 10/n tra le ascisse e con numero di punti nella partizione n=4:4:800. Utilizzare 10001 punti equispaziati nell'intervallo [0, 10] per ottenere la stima dell'errore. Che tipo di decrescita si osserva? Confrontare e discutere i risultati ottenuti, rispetto a quelli del precedente esercizio.

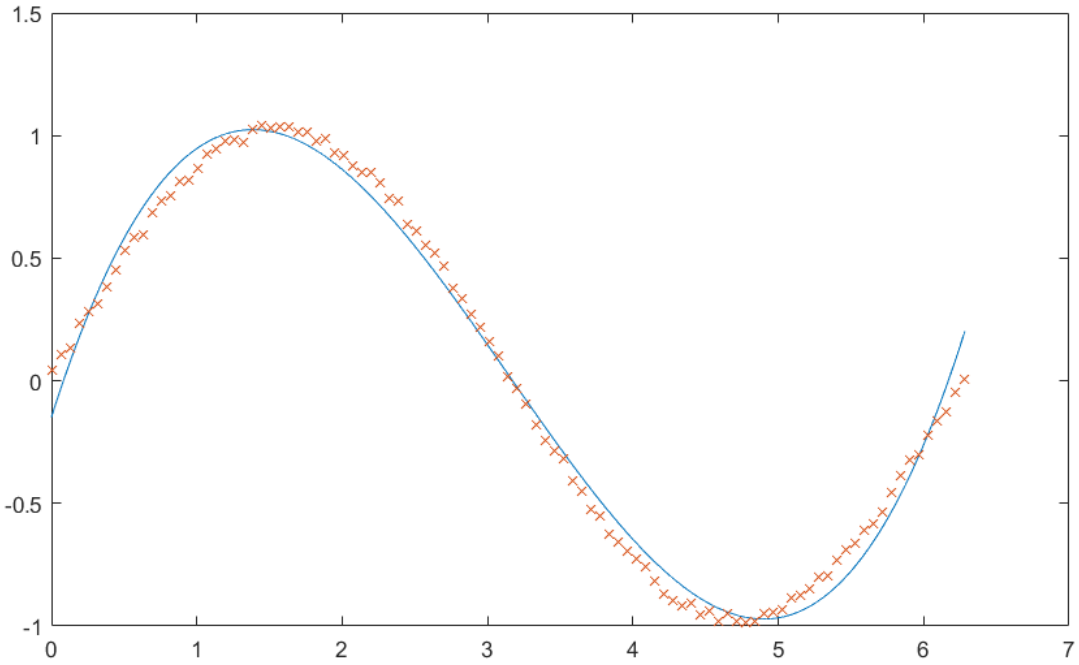
Possiamo osservare che mentre l'andamento della media dell'errore per lo spline cubico not-a-knot non subisce cambiamenti significativi quello dello spline cubico naturale è molto diverso, questo è dato dalle differenti condizioni aggiuntive che i due impongono oltre a quelle di interpolazione: nello spline cubico naturale, infatti, le derivate seconde agli estremi a e b vengono entrambi assunte uguali a zero, valore vicino per b (derivata seconda in b è 0.00058 circa) ma lontano per a (uguale a -2). Questo porta al peggioramento delle prestazione dello spline cubico naturale rispetto allo spline not-a-knot che impiega un metodo alternativo, più flessibile per determinare la derivata seconda agli estremi.



-Esercizio 27

Calcolare i coefficienti del polinomio di approssimazione ai minimi quadrati di grado 3 per i seguenti dati:
$>> rng(0);$
$>> xi = linspace(0, 2 * pi, 101);$
$>> yi = sin(xi) + rand(size(xi)) * .05;$
Graficare convenientemente i risultati ottenuti.

Ecco nella pagina successiva il grafico risultante che mostra il polinomio approssimante al terzo grado insieme ai punti rappresentanti i dati iniziali (eseguire scriptExe27).



-Esercizio 28

Costruire una function Matlab che, dato in input n, restituisca i pesi della quadratura della formula di Newton-Cotes di grado n. Tabulare, quindi, i pesi delle formule di grado 1, 2,. . . , 7 e 9 (come numeri razionali).

Function Matlab richiesto:

```
function c=ci(n)
%c=ci(n);
%ci restituisce un vettore c contenente i pesi di quadratura della
    formula
%di Newton-Cotes in base a n il grado di integrazione.
%Input:
%n-grado della formula di Newton-Cotes
%Output:
%c-vettore contenente i pesi richiesti
%release 25-04-2024
if n<1
    error("n illegale");
```

```
end
nn=n+1:-1:1;
c=zeros(1,n+1);
for i=0:n
    v=[0:i-1 i+1:n];
    den=prod(i-v);
    a=poly(v);
    a1=[a./nn 0];
    c(i+1)=polyval(a1,n)/den;
end
return
end
```

Listing 15: ci calcola dato n l'insieme dei pesi per la formula di Newton-Cotes di grado n.

I valori ottenuti sono riassunti nella seguente tabella:

c_{in}	n=1	n=2	n=3	n=4	n=5	n=6	n=7	n=9
i=0	1/2	1/3	3/8	14/45	95/288	41/140	1073/3527	130/453
i=1	1/2	1/4	9/8	64/45	125/96	54/35	810/559	1374/869
i=2	-	1/3	9/8	8/15	125/144	27/140	343/640	243/2240
i=3	-	-	3/8	64/15	125/144	68/35	649/536	5287/2721
i=4	-	-	-	14/45	125/96	27/140	649/536	704/1213
i=5	-	-	-	-	95/288	54/35	343/640	704/1213
i=6	-	-	-	-	-	41/140	810/559	5594/2879
i=7	-	-	-	-	-	-	1073/3527	243/2240
i=8	-	-	-	-	-	-	-	1374/869
i=9	-	-	-	-	-	-	-	130/453

-Esercizio 29

Scrivere una function Matlab,
$[If, err] = composita(fun, a, b, k, n)$
che implementi la formula composita di Newton-Cotes di grado k su n+1 ascisse equidistanti, con n multiplo pari di k, in cui:

- fun è la funzione integranda (che accetta input vettoriali);
- [a,b] è l'intervallo di integrazione;

- k, n come su descritti;
- If è l'approssimazione dell'integrale ottenuta;
- err è la stima dell'errore di quadratura.

Le valutazioni funzionali devono essere fatte tutte insieme in modo vettoriale, senza ridondanze.

Soluzione:

```
function [If,err]=composita(fun,a,b,k,n)
%[If,err]=composita(fun,a,b,k,n)
%composita stima l'integrale di fun in [a,b] usando la formula composita di
%Newton-Cotes di grado k su n+1 ascisse.
%Input:
%fun-funzione di cui si vuole stimare l'integrale definito nell'intervallo
%[a,b]
%a,b-estremi dell'intervallo
%k-grado di integrazione nei sottointervalli
%n-multiplo pari di k.
%release 26-04-2024
if nargin~=5
    error("numero di argomenti errato");
end
if a>b||mod(n,k)~=0
    error("argomenti errati");
end
if mod(n,k)~=0
    error("n illegale");
end
%calcolo delle ascisse e delle corrispondenti ordinate
xi=linspace(a,b,n+1);
yi=feval(fun,xi);
%calcolo dei pesi di quadratura per la formula di Newton-Cotes
kk=k+1:-1:1;
c=zeros(1,k+1);
for i=0:k
    v=[0:i-1 i+1:k];
    den=prod(i-v);
    ap=poly(v);
    ap1=[ap./kk 0];
    c(i+1)=polyval(ap1,k)/den;
end
%calcolo dell'integrale composito di Newton-Cotes di grado k
```

```
35 If=0;
36 for i=1:n+1
37     ind=mod(i-1,k);
38     if ind==0&&i~=1&&i~=n+1
39         If=If+2*yi(i)*c(ind+1);
40     else
41         If=If+yi(i)*c(ind+1);
42     end
43 end
44 If=If*((b-a)/n);
45 %calcolo della stima d'errore
46 p=n/k;
47 If2=0;
48 for i=1:p:n+1
49     ind=mod((i-1)/p,k);
50     If2=If2+yi(i)*c(ind+1);
51 end
52 If2=If2*((b-a)/k);
53 if mod(k,2)==0
54     err=abs((If-If2)/(p^(k+2)-1));
55 else
56     err=abs((If-If2)/(p^(k+1)-1));
57 end
58 return
59 end
```

Listing 16: composita calcola la stima dell'integrale della funzione fun nell'intervallo specificato usando la formula di Newton-Cotes composita di grado k

-Esercizio 30

Calcolare l'espressione del seguente integrale:

$$I(f) = \int_0^1 e^{3x} dx$$

Utilizzare la function del precedente esercizio per ottenere un'approssimazione dell'integrale per i valori k = 1, 2, 3, 6, e n = 12. Tabulare i risultati ottenuti, confrontando l'errore stimato con quello vero.

Eseguendo lo script scriptExe30 otteniamo i seguenti risultati:

Tab	Integrale stimato	Errore stimato	Errore vero
k=1	6.39494578983666	0.0290057529493509	0.0331001487740998
k=2	6.36198268140859	0.000108159216999796	0.000137040346029238
k=3	6.36215172028801	0.000250165168163046	0.000306079225458333
k=6	6.36184573734026	7.76640488112156e-08	9.62777075841359e-08

Possiamo osservare che l'errore stimato segue in modo vicino il valore dell'errore vero, e entrambi diminuiscono velocemente al crescere del grado di integrazione. Per calcolare la stima dell'errore data dalla formula composita di Newton-Cotes ho usato la formula:

$$I(f) - I_{k,pk}(f) \approx \frac{I_{k,pk}(f) - I_k(f)}{p^{k+i} - 1}$$

dove

1. i=1 se k è dispari i=0 se è dispari
2. k è il grado dell'integrazione
3. p=m/n, con m numero di punti nell'integrazione composita meno 1

Questa formula è stata ricavata come generizzazione di:

$$I(f) - I_{1,2}(f) \approx \frac{I_{1,2} - I_1(f)}{3}$$